



Designing headless components based on inversion of control:

`<TimeSeriesChart />`

🕒 Created	@January 18, 2022 11:44 AM
👤 Owner(s)	 Mithi Sevilla
☰ Summary	How to use ` <code><TimeSeriesChart /></code> ` and ideas on how we can design charts for the future....
🔗 URL	
☰ Property	

TLDR:

Please checkout the storybook examples of this MR: https://gitlab.com/luxorlabs/lmf-v5/-/merge_requests/699/ to see how `<TimeSeriesChart />` is intended to be used. Eager

to hear your comments, suggestions, or violent reactions 😊.

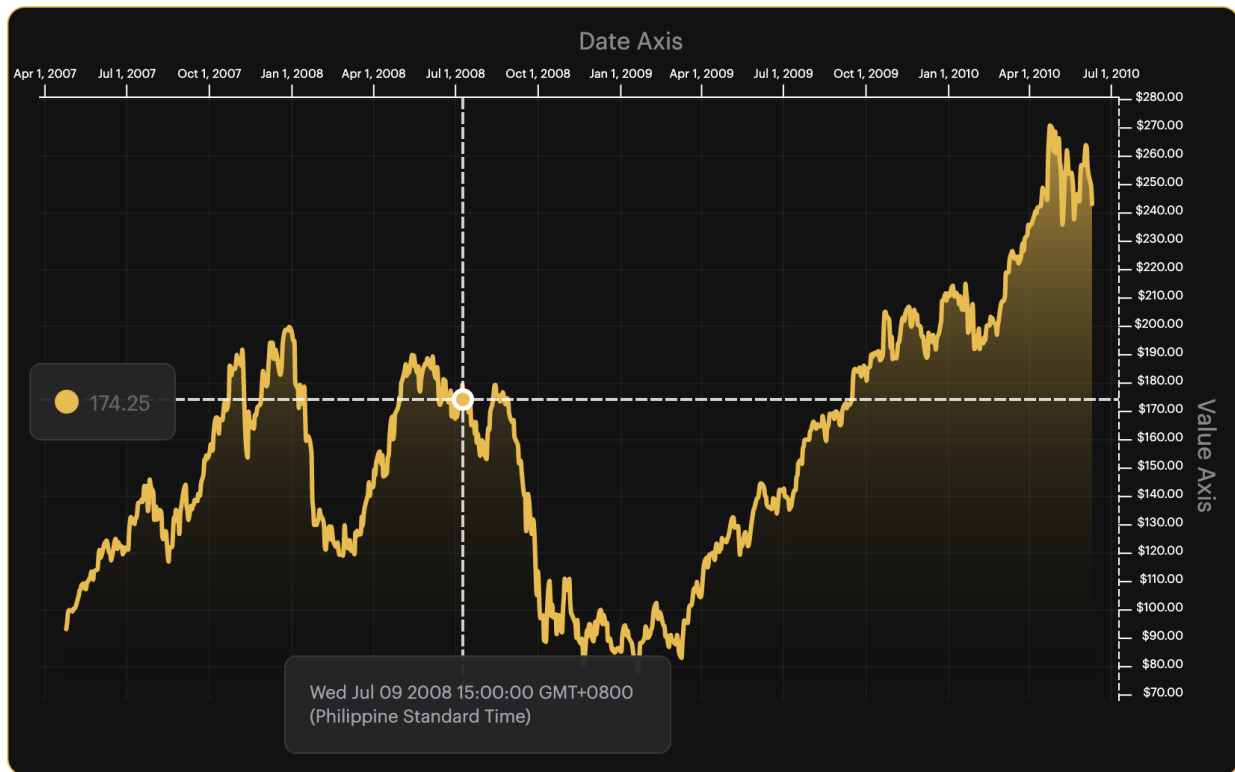
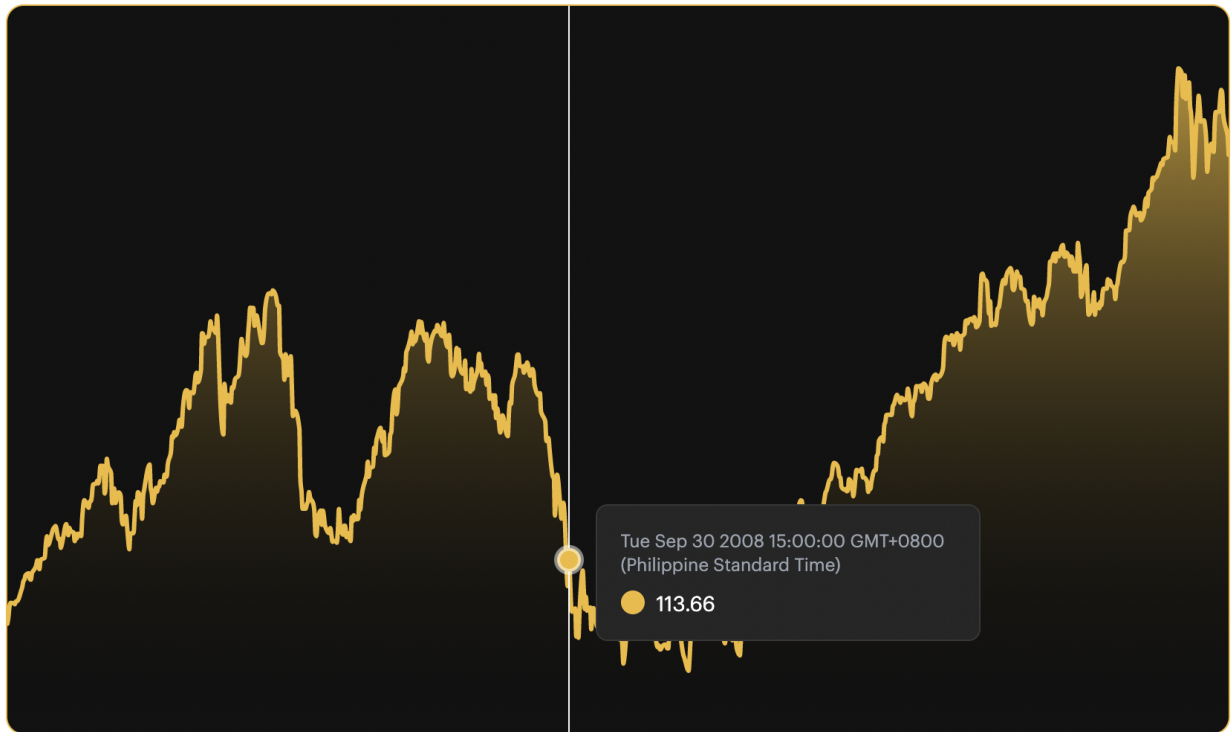
An 18 second video is worth a thousand words.

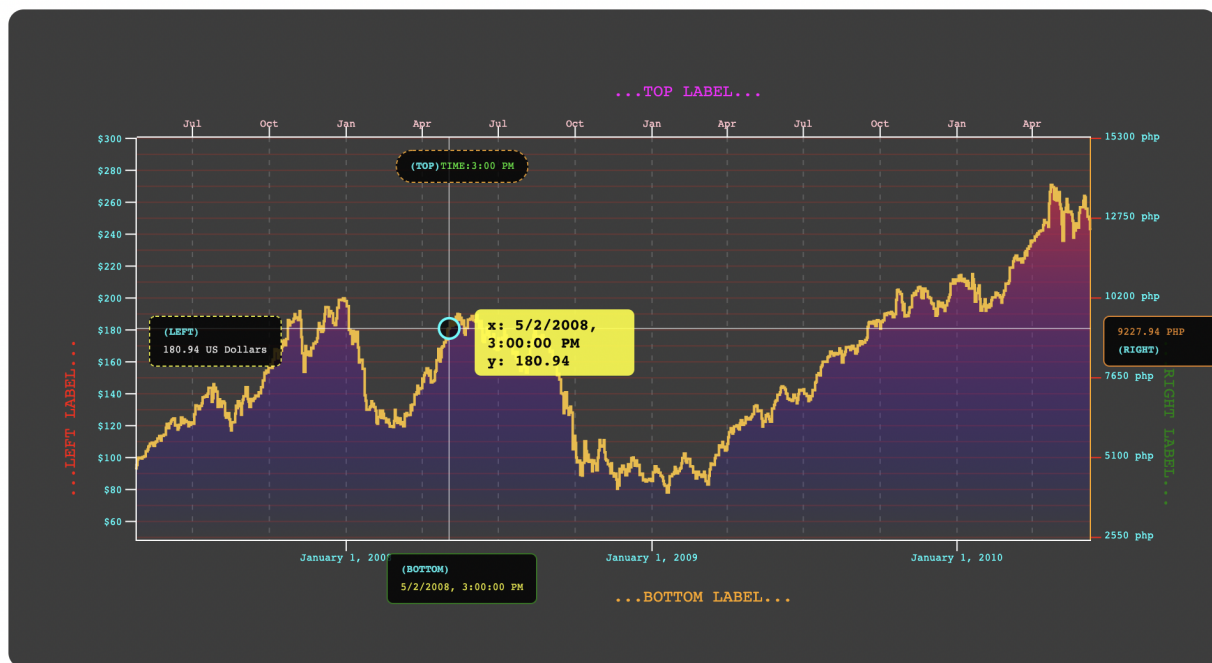
<https://s3-us-west-2.amazonaws.com/secure.notion-static.com/7e848eb6-0a33-410d-b969-cf1b9cf45aa9/partial-charts.mov>

It's responsive to its parent container. It also takes the background of its parent.

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/add708a1-3153-4b72-9552-389cb7b195b5/responsiveness_demonstration.mov

Some demos illustrating what you can do.





The last example looks funky because its rendering using all the visualization primitives available inside the component. Just demonstrating how customizable it is.

Intro: Goals and Reasons

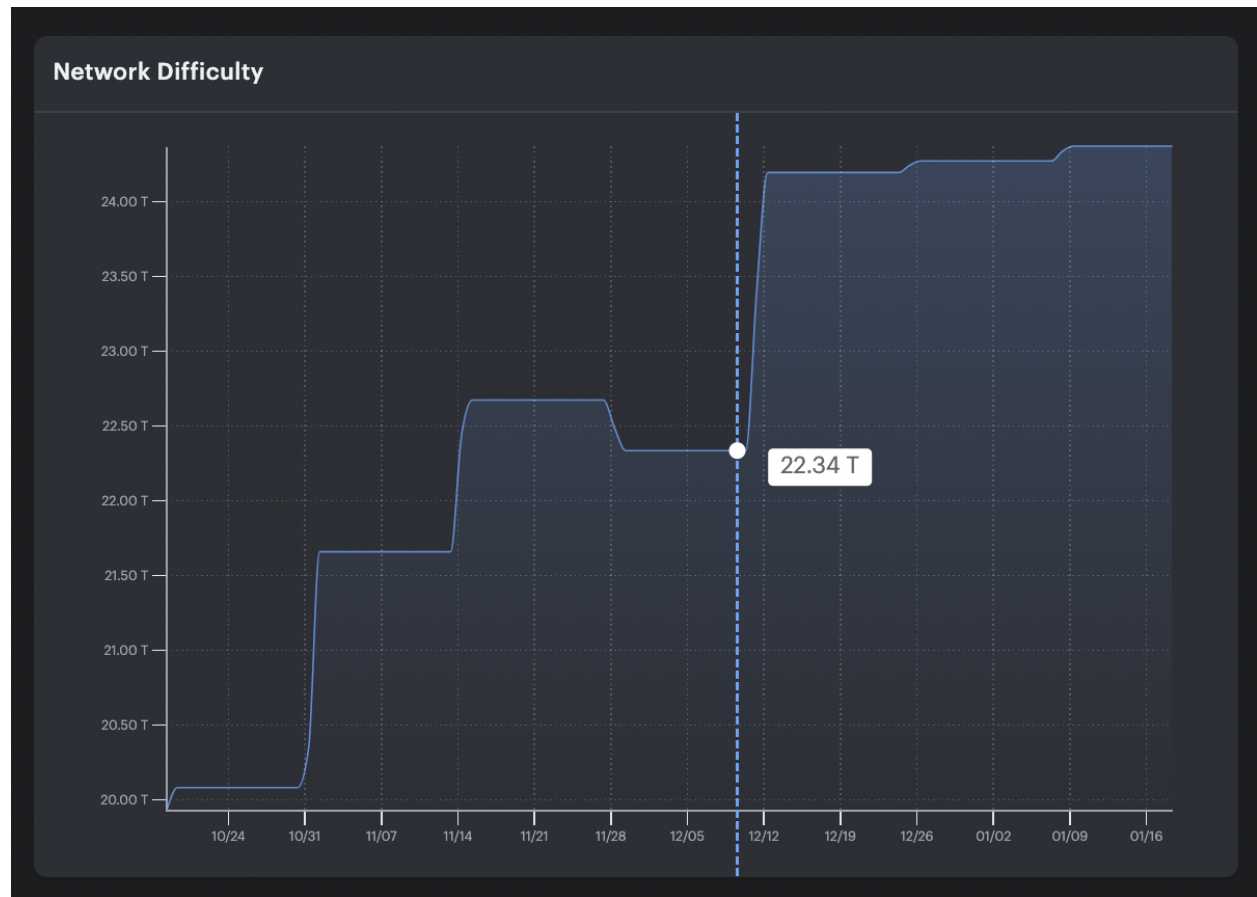
The goal is to build a `<TimeSeriesChart />` component that is easy to reuse, customize, and style. Additionally, the component should be as performant as we need it to be and relatively easy to maintain, debug, and extend. The principles discussed here can be kept in mind when creating other reusable charts for the future.

Why do this? We've built so many time series charts in the past, and some of them we've been able to reuse a few times.. but most of the time what we end up doing is for each new chart requirement, we build each from scratch... copy pasting existing charts components and then adding, removing, and modifying visualization primitives to suit our specific needs. This practice, I believe, was actually the best way to do it at those particular times, because *duplication is far cheaper than the Wrong Abstraction (Sandi Metz)*, (Avoid hasty abstractions - KCD) and quoting from from Google Engineering Practices :

The future problem should be solved once it arrives and you can see its actual shape and requirements in the physical universe.

But now, I think we can already see its actual shape and requirement in the physical universe. 😊

Here are some of the many time series charts we have in our apps:





hashrateindex.com

Hash Price

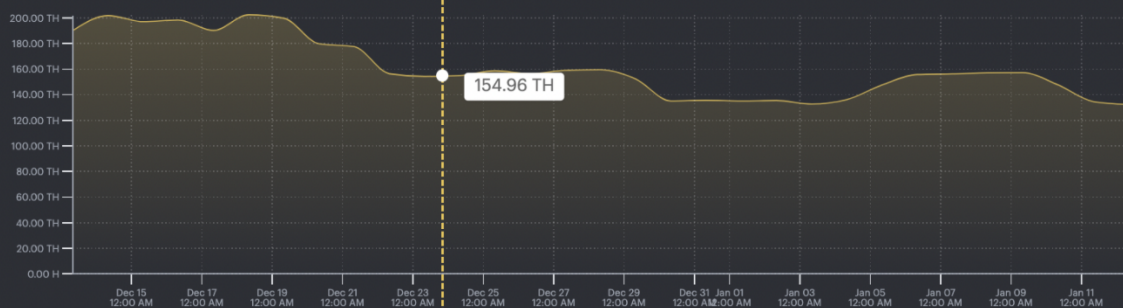
Network Data

The Bitcoin Mining Pools page displays the latest blocks and total hashrate of top mining Bitcoin mining pools.



VIEW ANALYSIS

24h (30d period)

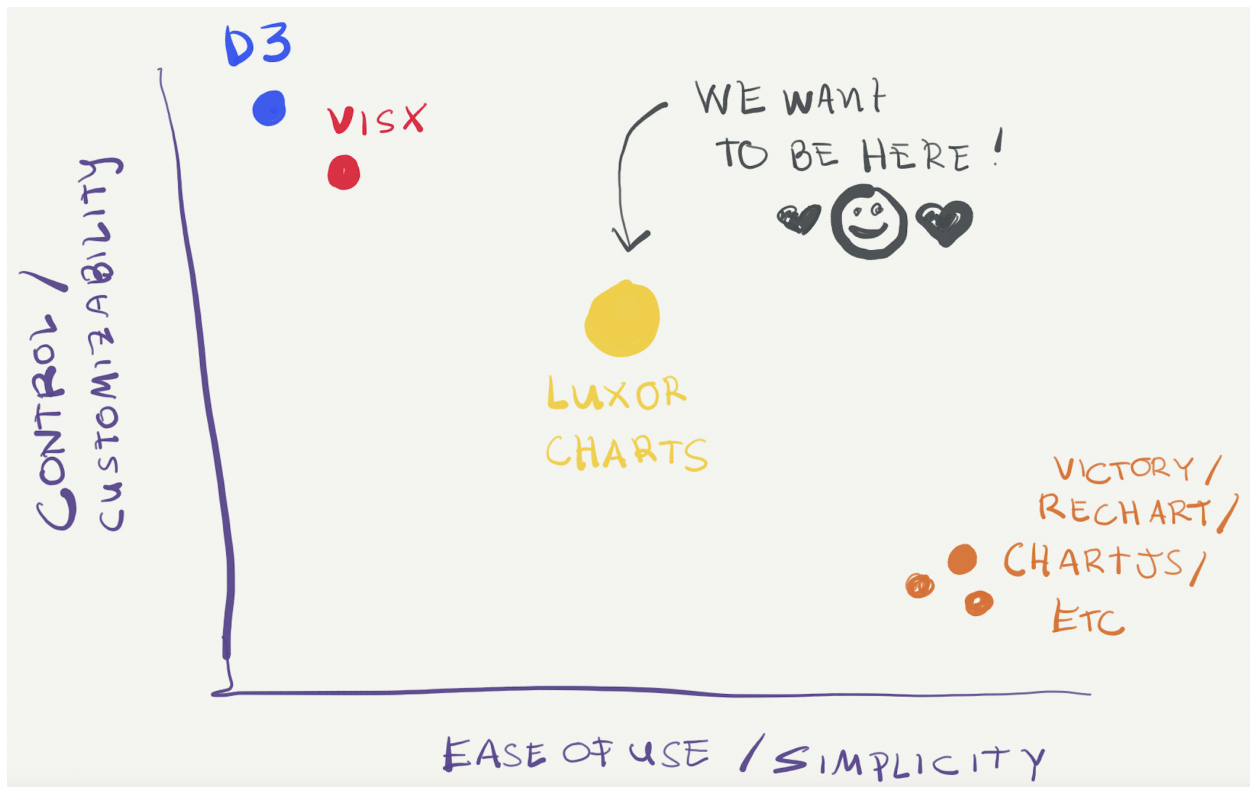




We chose to build on top of Airbnb's Visx, so we have complete and full control... As a result, we've built some really amazing data visualization that our users will enjoy looking at, and interacting with. Good job everyone! 🎉

However, having low-level control comes with an unfortunate cost... development effort/time/happiness and ease-of-use.

I think the new goal is to strike a good balance between control / customizability and ease-of-use/development effort/time/happiness.



Strategy / Approach

The key concepts:

1. Use the **Render Props** technique, a form of inversion of control.
2. **API first.** Draw inspiration from RDD **Readme Driven Development** (Not religiously).
3. **Do not render unless specified all of the time** (Except for a few tiny exceptions).
Less is more. If you don't specify props of an specific (and available) Visx visualization primitive (e.g styles of `<GridColumns />` , `<LinearGradient>` etc) then this primitive will not be rendered. Concretely, if you don't specify a render function for the tooltip, then that tooltip will not be rendered. In Filipino terms, **`Ayokong nagmamamarunong ang component ko`** (As Mon, Carl, Kevin, or CJ what this means)

4. Reusing components means importing themes. Almost completely headless.

The component will not make default design decisions for you every time (except for one exception). If you don't want to think, import default props from an existing file if you want to reuse. If most can be reused but not all, you can just and override those.

1. RenderProps

Render props is the technique of passing a prop to a component wherein the prop is a function that returns how something will be displayed. Example:

```
<ChartWithCursor
  cursorRenderFunction={({position}) => {
    return (
      <div style={{color: 'red', background: "black" }}>
        I am displayed where the cursor is {position.x}, {position.y}
      </div>
    )
  }}
/>
```

The concrete example for `<TimeSeriesChart />` will be on section 3.

2. API First

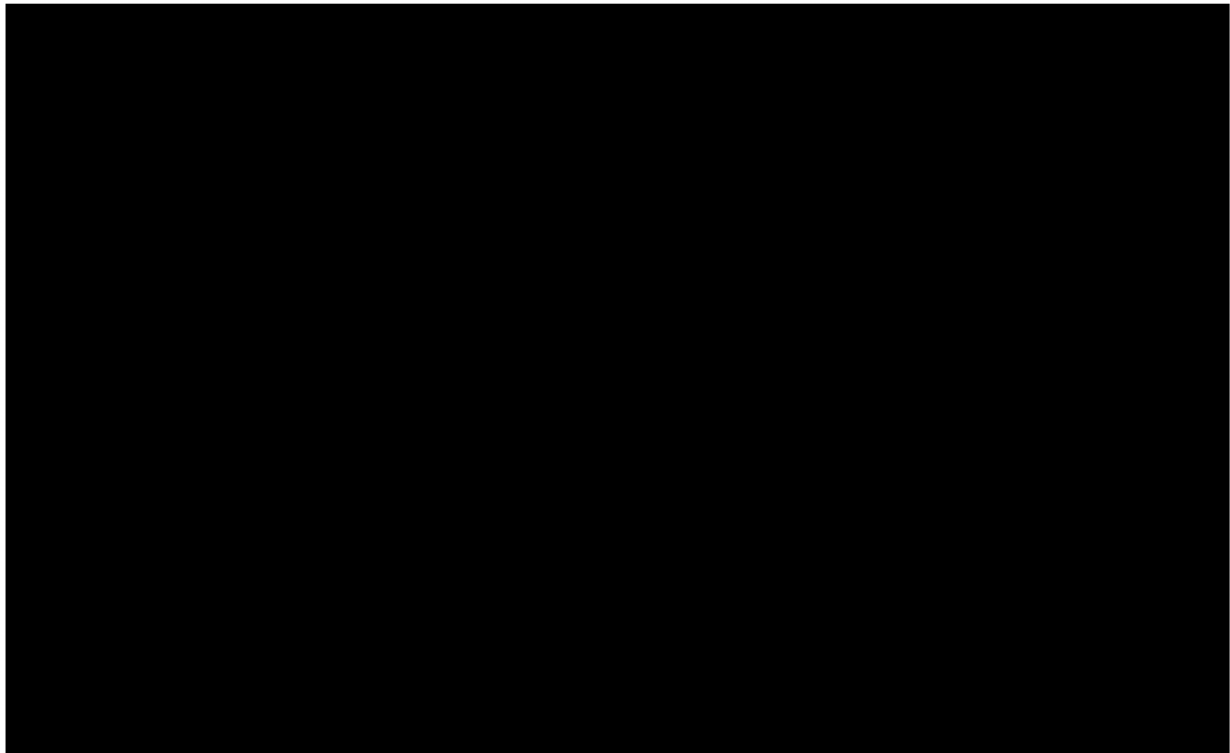
Writing API is hard. But RDD can help. At its core RDD is beginning with the end in mind. We will write first how our component will be used (the API) before actually writing the implementation. Idea (3) and (4) are actually design decisions I made as a result of thinking API first.

So, how will my component be used?

At the bare minimum, I want the component to always have the size and background of its parent container. You can render the component without passing props and will display nothing.

```
<div tw="w-full h-3/4 bg-black">
  <TimeSeriesChart />
</div>

// is the same as
<div tw="w-full h-3/4 bg-black">
  <TimeSeriesChart data={[]} />
</div>
```



The data I would pass to `<TimeSeriesChart />` is simply of type `{date: Date, value: number}` `[]` and should be sorted by date.

Here are just a few exceptions. At the very least, I want to see something when I pass data (line will have `strokeWidth=1` and `color='hsl(43, 86%, 57%)'` - a default color that can be seen in most light and dark background), with a curve type of `monotoneX`. Why? Because I think this is the intuitive thing behavior that the user expects. When the user passes data to a chart component, the user most likely be puzzled if s/he sees a blank

screen. Also, there are some default behaviour when the `loading = true` prop is passed because of the same reason. Quoting from **Rusty's API Design Manifesto**: One characteristic of a good API is when: The obvious use is (probably) the correct one.

Thase is the only time that the component will make a style decision for you.

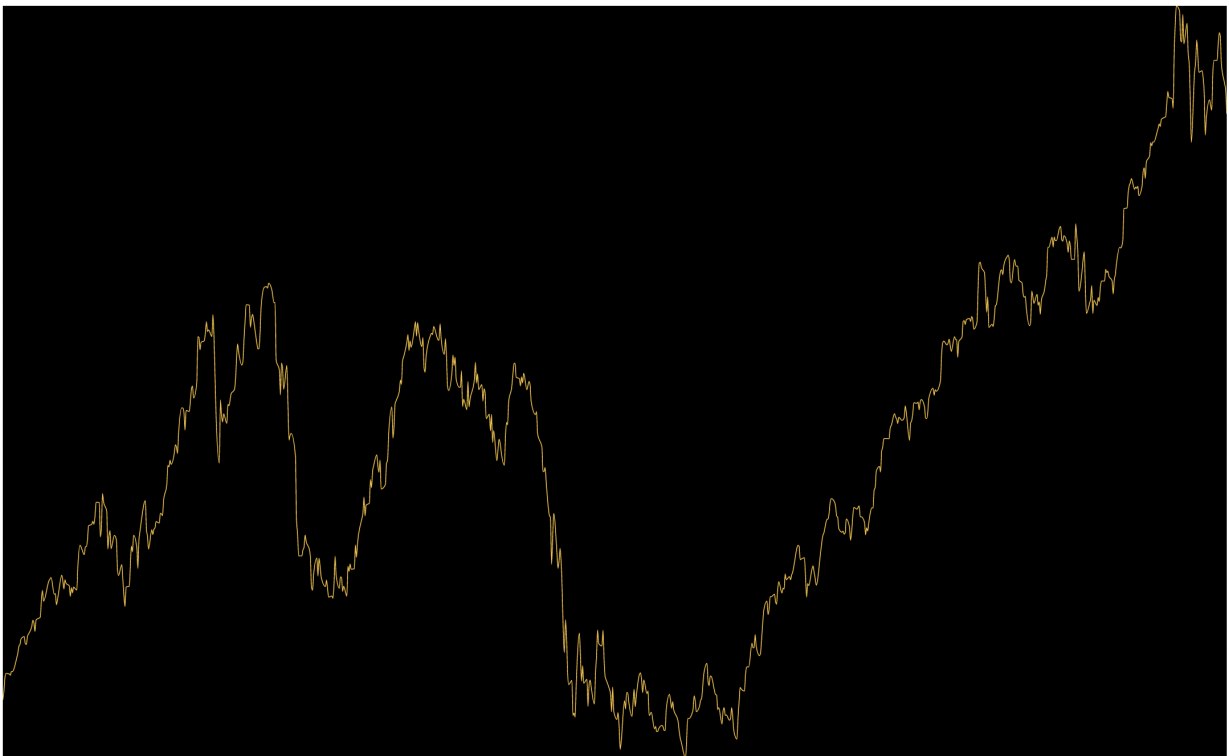
(Note: These defaults are, of course, something you can easily override.)

Example:

```
const arraySortedByDate: {date: Date, value: number}[] = /* my sorted data here */

<div tw="w-full h-3/4 bg-black">
  <TimeSeriesChart data={arraySortedByDate} />
</div>

// will yield something like the graph below
```



```
<div tw="w-full h-3/4 bg-black">
  <TimeSeriesChart data={arraySortedByDate} loading/>
</div>
```

Open gif below

<https://s3-us-west-2.amazonaws.com/secure.notion-static.com/fd8910a6-62bb-47ef-ab61-8d6638434392/loading-chart-gif.gif>



here's another example

3. Do not render unless specified.

This is best illustrated by the 9 second video below:

<https://s3-us-west-2.amazonaws.com/secure.notion-static.com/888a21db-53e5-4dba-8772-e82277644c40/basic.mov>

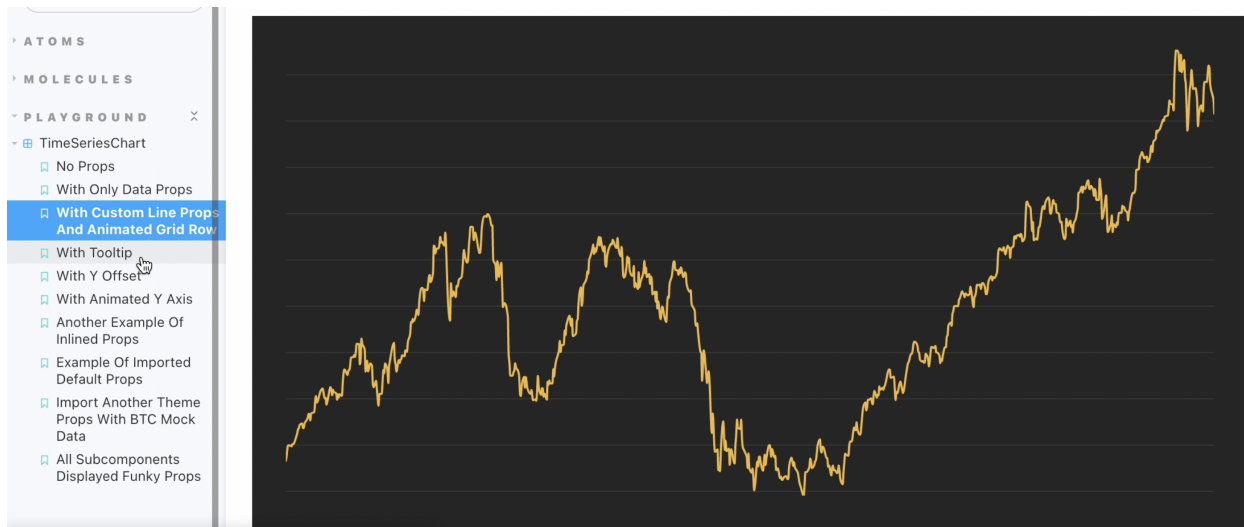
Given our bare minimum `<TimeSeriesChart data={arraySortedByDate} />` from the previous section's last example. I can built on top of that.

Suppose I want to style the line itself, like I want it to be thicker and change its color. Suppose I also want to add grid row lines that has a little animation on mount.

It's easy as doing this:

```
<div tw="width[95vw] height[95vh] bg-box-lighter p-8">
  <TimeSeriesChart
    lineProps={{
      path: { stroke: theme`colors.primary`, strokeWidth: 2 },
    }}
    gridProps={{
      row: { stroke: 'gray', strokeOpacity: 0.2 },
    }}
    animated={{ gridRows: 'center' }}
    data={arraySortedByDate}
  />
</div>
```

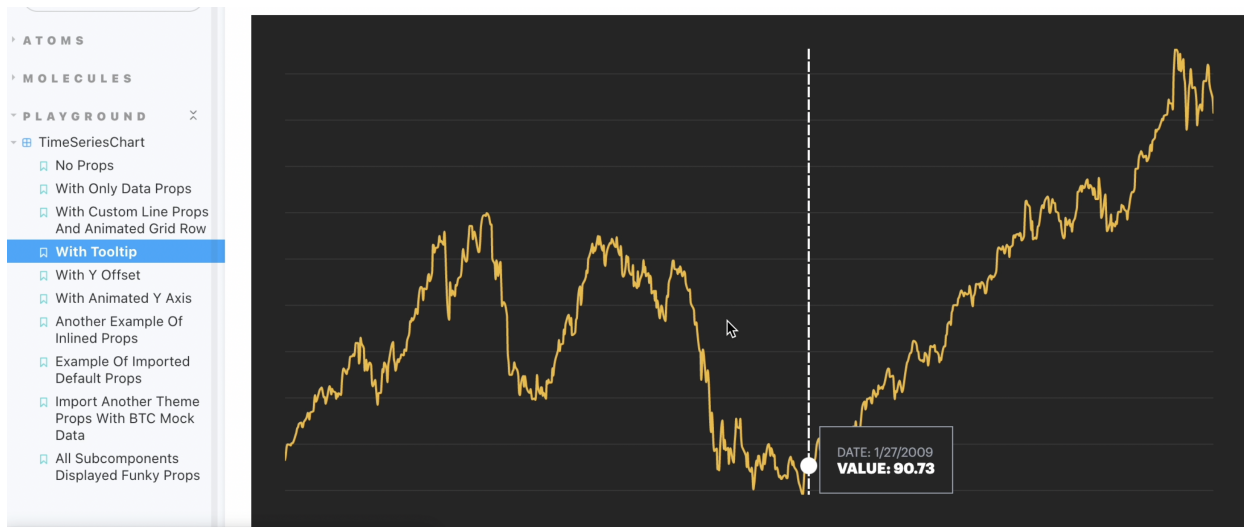
Here's a screenshot of how it will look.



Building on top of that, suppose I want to add a tooltip. I only want to a tooltip to pop-up near the current data point (`bound`), I don't want to display the tooltip x-axis (horizontal line). Just the tooltip y-axis (vertical line).

Given this, I just need to specify the styles of the tooltip y axis, as well as a `render prop` to specify how the tooltip pop up looks like e.g:

```
<TimeSeriesChart
  customTooltip={{
    renderFunctions: {
      bound: (tooltipData) => (
        <div tw="-mt-12 p-4 border border-gray-400 bg-box-lighter">
          DATE: {tooltipData.date.toLocaleString()}
          VALUE: {tooltipData.value.toFixed(2)}
        </div>
      )
    },
    props: {
      lineY: { stroke: 'white', strokeWidth: 2, strokeDasharray: '10,2' },
      currentPoint: { fill: 'white', r: 8 },
    },
  }}
  {/**** All the other props earlier **** /}
/>
```



By default, the edges of the chart are bounded by the minimum and maximum values of the data array that is passed. Concretely for the x-axis, the domain is from the minimum date to the maximum date given the data passed..

We can override this so that it doesn't feel too cramped..

```
import { sub, add } from 'date-fns';

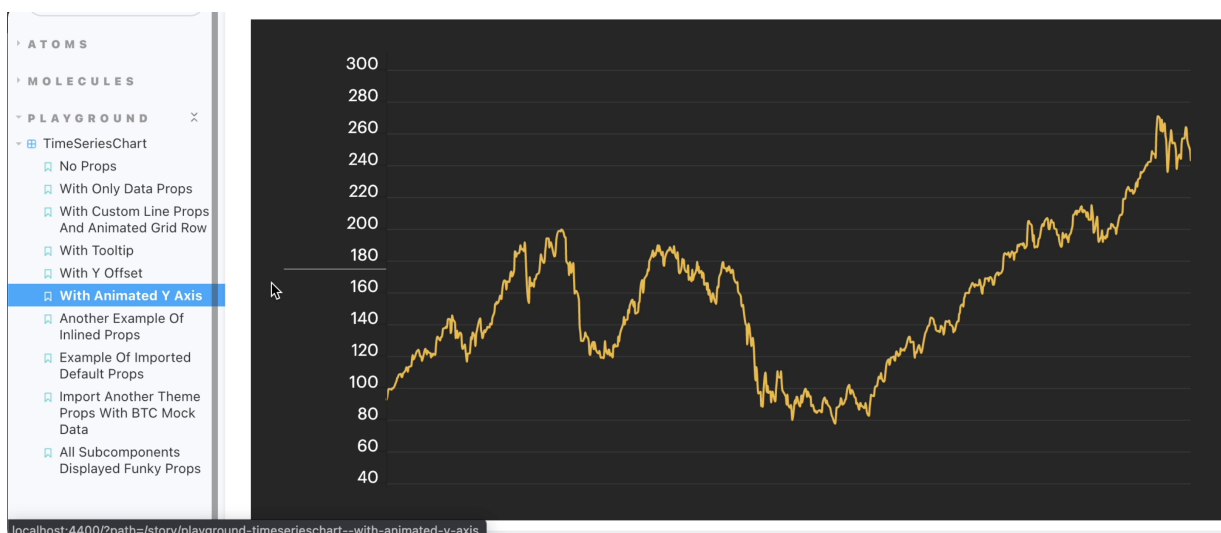
<TimeSeriesChart
  scaleOffsetFunctions={{
    domain: {
      getMaximumDate: (date) => add(date, { months: 2 }),
      getMinimumDate: (date) => sub(date, { months: 2 }),
    },
    range: {
      getMaximumValue: (val) => val + 40,
      getMinimumValue: (val) => val - 40,
    },
  }}
  {/**** All the other props earlier **** /}
/>
```

I can also display the `yAxis` if I want, and make that `yAxis` animate on mount...

```

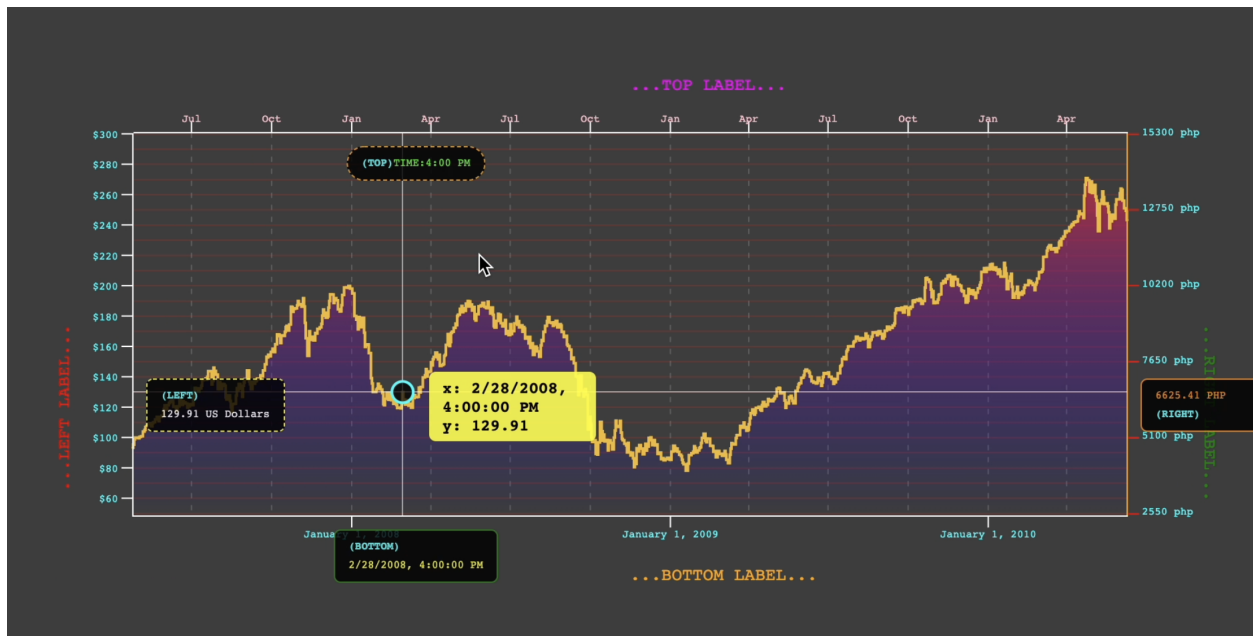
<TimeSeriesChart
  padding={{ left: 100, right: 0, top: 0, bottom: 0 }}
  axisProps={{
    left: {
      tickLabelProps: () => ({ textAnchor: 'end', fill: 'white' }),
      tickFormat: (n: number) => n.toString(),
    },
  }}
  animated={{ gridRows: 'center', xAxis: 'min', yAxis: 'min' }}
  {/**** blah **** /}
/>

```



And so on and so forth. You get the idea. If you don't provide props for that visualization primitive, that visual primitive will not be rendered. I think this is preferable, because it's easier to add things than remove things. *Simple is better than complex*. Always err on the side of simplicity. 😊

Below is an example showing all the visual primitives that you can show inside the `<TimeSeriesChart />`. It looks ugly yes, but that's not the point. I'm just demonstrating how customizable it is....



4. Reusing components means importing themes.

It would be a pain to specify all the possible props each time you render a `<TimeSeriesChart />`. Since the props (other than the data) is usually just static, it would be best to NOT declare it inline. Instead, declare it OUTSIDE the component. This minimizes unnecessary re-renders and hopefully making this chart more performant as a side effect. Given the nature of React itself, since all the props are constant aside from the data, the chart will change only when the data changes or when its parent component re-renders.

Examples:

```
import { darkThemeProps, lightThemeProps } from 'timeSeriesChartThemes'

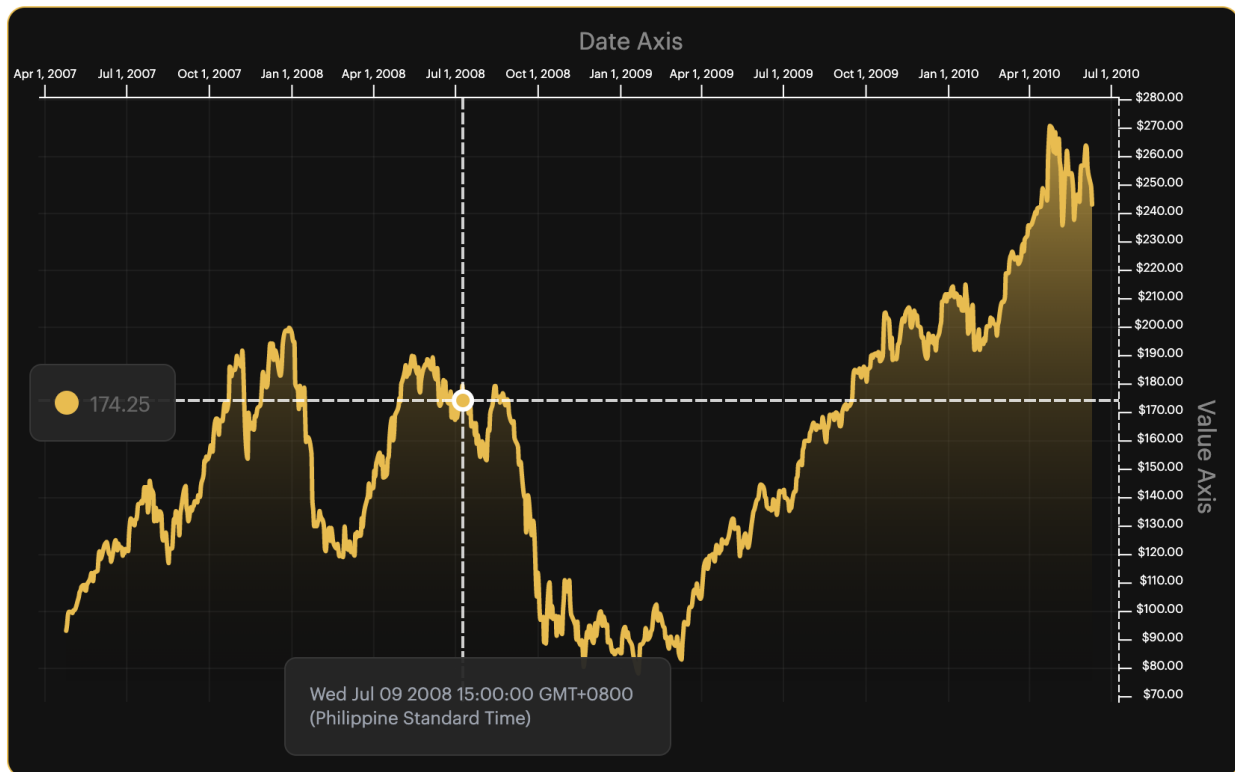
const btcData = bitcoinPrice.prices.slice(0, 100).map((data) => {
  return { date: new Date(data.time), value: Number(data.price) || 0 }
}).sort((a, b) => { return a.date > b.date ? 1 : -1}})

/* Example 1: */
<TimeSeriesChart data={btcData} {...darkThemeProps} />

/* Example 2: overrides left padding*/
```

```
const EX2_PROPS = {...darkThemeProps, padding: { ...darkThemeProps.padding, left: 0 } }
<TimeSeriesChart data={btcData} {...EX2_PROPS} />

/* Example 3: Use light theme props, but the light theme
   props shows a left tooltip that we don't want to show */
const EX3_PROPS = {...lightThemeProps, customTooltip: { renderFunction: { left: undefined }}}
<TimeSeriesChart data={btcData} {...EX3_PROPS} />
```





Ending Remarks

That's pretty much it. Hoping to hear your thoughts, whether or not you think this is a good or bad idea and why. Is the kind of chart component is useful to you and if so do you think you'll be happy reusing in the future? Why or why not? Please let me know if I should proceed to building an equivalent `<MultiLineTimeSeriesChart />` component or other charts or if you think there are other tasks that I should prioritize working on. Please let me know. I'm really looking forward to hearing your thoughts! Thanks. **-Mithi**